Fundamentals of Database Design

From business requirements to reliable relational systems

- Model real requirements
- Relationships and integrity
- Keys, performance, and validation

Practical Web Database Design

Chapter 4: Fundamentals of Database Design

Presented by Abdalla Fadl

Full-Stack Java Developer

youtube.com/c/AbdallaFadl

02 Why model before building?

01 Clarify the business requirements

02 Discover missing cases before implementation

03 Share a design that developers and customers can review

03 Modeling is iterative

01 Gather requirements and create a model

02 Review with users; refine the assumptions

03 Repeat, then implement the validated design

04 From business model to physical database

01 Conceptual model: business concepts

02 Logical model: data structure and relationships

03 Physical design: platform-specific implementation

05 The three elements of an ER Model

01 Entities: things we describe

02 Attributes: properties we record

03 Relationships: associations between things

06 Read the entity box

01 PK identifies the primary key

02 FK identifies a foreign key constraint

03 Attribute names and types describe the columns

07 Cardinality and optionality

01 Maximum: one or many

02 Minimum: zero or one

03 Read each end from the opposite entity

08 Four relationship-end meanings

Minimum	Maximum	Meaning
0	1	Zero or one
1	1	Exactly one
0	Many	Zero or many
1	Many	One or many

KEY TAKEAWAY: Read the minimum and maximum at both ends.

09 One-to-Many: categories and products

01 A category can contain many products

02 Each product has one category in this example

03 Store CategoryID on Products, the many side

10 One-to-Many: customers and orders

01 A customer can place multiple orders

02 Each order belongs to one customer

03 Customers may exist before placing an order

11 Many-to-Many needs an association

01 One order contains many products

02 One product appears in many orders

03 OrderItems turns this into two One-to-Many links

OrderItems: data in the relationship

OrderID	ProductID	Quantity	PurchasePrice
1	4	2	100
2	4	1	100
2	7	3	50

KEY TAKEAWAY: Association attributes belong in the junction table.

13 Composite keys in a junction table

01 PRIMARY KEY (OrderID, ProductID)

02 Each component may repeat separately

03 The complete pair must be unique

14 One-to-Many Recursive

01 Categories references Categories

02 Each node has zero or one parent

03 Each parent can have many children

15 A category hierarchy in rows

CategoryID	CategoryName	ParentCategory
1	Computers	NULL
2	Databases	1
3	Database Design	2

KEY TAKEAWAY: The parent reference points back to the same table.

16 A foreign key does not prevent cycles

01 Reject a node as its own parent

02 Reject any descendant as the new parent

03 Existing IDs can still form an invalid tree

17 Traverse a hierarchy

01 Top-down: roots to descendants

02 Bottom-up: node to ancestors

03 Depth controls display indentation

18 Correct the starting query

```
roots = query("SELECT * FROM Categories
               WHERE ParentCategory IS NULL")
traverse(nodes, depth):
  for node in nodes:
    print(node.name, depth)
    children = findChildren(node.id)
    traverse(children, depth + 1)
```

KEY TAKEAWAY: Start at roots; an empty child set ends that branch.

19 Recursive queries have a cost

01 One initial query plus one per visited node

02 N nodes can produce N + 1 queries

03 Fetch flat rows once or cache a tree

20 Many-to-Many Recursive: bundles

01 A bundle is itself a Product

02 A product can belong to many bundles

03 Both junction-table FKs reference Products

21 Bundles can contain bundles

01 Bundle 10 contains products 3 and 4

02 Bundle 11 can contain bundle 10

03 Prevent direct and indirect containment cycles

22 Actual One-to-One

01 Each department has one manager

02 Each manager manages at most one department

03 Use a foreign key with UNIQUE

23 Which side owns the foreign key?

01 Store the association once

02 Either side is possible in One-to-One

03 Placement affects mandatory participation

24 Super-type and Sub-type

01 Super-type stores shared attributes

02 Sub-type stores specialized attributes

03 Subtype ID can be both PK and FK

25 Items and Files: one shared tree

01 Items stores name and parent relationship

02 Files stores size and binary content

03 A nullable FileID distinguishes folders in the example

26 Integrity belongs in the database

01 Entity integrity: valid identifiers

02 Domain integrity: valid attribute values

03 Referential integrity: valid references

27 Primary key rules

01 Single key: unique and NOT NULL

02 Composite key: unique complete combination

03 No composite-key component may be NULL

28 Natural keys: test the business meaning

01 Author + Title + Publisher can collide

02 ISBN identifies an edition/format, not a copy

03 Current sample uniqueness is not a guarantee

29 Surrogate keys: stable identity

01 Generated numeric ID or GUID/UUID

02 Independent of business attribute changes

03 Keep business UNIQUE constraints where needed

30 Choose keys without absolutes

01 Generated numbers can have gaps

02 Random UUIDs make guessing harder

03 Timestamps alone do not ensure uniqueness

31 When a surrogate adds little

01 UserGroup can use (UserGUID, GroupID)

02 An extra ID does not prevent duplicate membership

03 Add a separate ID only for a concrete need

32 Define the attribute domain

01 Type and capacity

02 Constraints and optionality

03 Defaults describe omission behavior

33 Integer and text choices

01 Signed 32-bit: -2147483648 to 2147483647

02 CHAR is fixed length; VARCHAR is variable length

03 Phone numbers are text, not quantities

34 DECIMAL: precision and scale

01 Precision = total digits

02 Scale = fractional digits

03 DECIMAL(9,4): five integer and four fractional digits

35

Price precision matters

Value / quantity	Meaning
$0.001	One tenth of a cent
4,000,000 units	Holding quantity
$4,000	Total value difference

KEY TAKEAWAY: Small per-unit differences multiply at high volumes.

36 Temporal and Boolean requirements

01 Store DOB; calculate current age

02 Purchase time requires date and time

03 No risk and not assessed may differ

37 NULL, zero, and missing information

01 NULL: price not recorded

02 0: recorded price is zero

03 Use IS NULL and IS NOT NULL

38 DEFAULT and NOT NULL are separate

```
-- SQL Server example
CREATE TABLE Example (
  Status VARCHAR(20) NOT NULL DEFAULT 'Pending'
);

-- DEFAULT supplies; NOT NULL requires.
```

KEY TAKEAWAY: An explicit NULL is different from an omitted value.

CHECK constraints and UNKNOWN

PricePaid	Condition >= 0	Nullable CHECK
10	TRUE	Accepted
0	TRUE	Accepted
-1	FALSE	Rejected
NULL	UNKNOWN	Accepted

KEY TAKEAWAY: CHECK rejects FALSE; NOT NULL prohibits missing values.

40 Referential integrity and orphan rows

01 FK references an existing suitable key

02 Insert parent before a required child

03 FK may target a suitable UNIQUE key, not only PK

41 Insert, update, and delete violations

01 INSERT: missing referenced parent

02 UPDATE: old key remains referenced

03 DELETE: children outlive the referenced parent

42 Choose the referential action

Action	Parent key update	Parent deletion
RESTRICT	Prevent violation	Prevent violation
CASCADE	Update child FKs	Delete children
SET NULL	Clear child FKs	Clear child FKs

KEY TAKEAWAY: Choose actions from the business lifecycle.

43 CASCADE follows the business lifecycle

01 Order deletion may remove OrderItems

02 Employee deletion should not erase Projects

03 One statement can still perform substantial work

44 Denormalization: faster reads, more upkeep

01 Store Publisher.BookCount instead of recounting

02 Update derived values when source data changes

03 Measure the bottleneck before duplicating data

45 Keep derived data consistent

01 Immediate updates need atomic coordination

02 Concurrent writes must not lose changes

03 Background summaries trade freshness for latency

46 CASE tools: model versus diagram

01 Database-aware tools understand keys and types

02 A model can contain multiple focused diagrams

03 Technical validation is not business validation

47 Forward, reverse, and round-trip engineering

01 Forward: model to database definition

02 Reverse: database to model

03 Round-trip: compare and manage both directions

48 Three layers of sample data

01 Client examples validate requirements

02 Valid and invalid rows test constraints

03 Realistic volume and distribution test performance

49 Load testing versus stress testing

01 Load: expected production demand

02 Stress: exceed expectations and observe limits

03 Measure latency, errors, and recovery

50 Interview answer: design a relationship

01 State the rule in both directions

02 Separate maximum cardinality from optionality

03 Explain FK location and enforcement

51 Project use: a practical design review

01 Confirm what each row represents

02 Review keys, constraints, cycles, and deletion rules

03 Test correctness before optimizing measured hotspots

52 Summary and key takeaway

01 Model the business; validate with real examples

02 Enforce identity, domains, and references

03 Optimize measured workloads with explicit trade-offs

KEY TAKEAWAY: Make rules explicit, enforce them, and validate with realistic data.

Created with AI assistance. Reviewed and presented by Abdalla Fadl.